UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623
Database Programming
2025/2026 SEMESTER 1
Lab Assignment 2

Name	:	WOO CHENG SHUAN
Matric Id	:	A23CS0283
Section	:	02

Academic Integrity

All students must work INDIVIDUALLY. Use of AI-generated SQL or code sharing is strictly prohibited. Identical or AI-generated scripts will receive **zero marks.**

Provide screenshots of every instruction in all tasks.

SUBMISSION REQUIREMENTS

Submit one (1) ZIP file containing:

1. Lab2_Clinic_Matric Number.txt (Copy from MongoDB Shell - Mongosh)
    - All MongoDB commands used
    - Clearly labelled by question number
2. Lab2_Clinic_Matric Number.pdf
    - Screenshots of command outputs
    - Each screenshot must clearly show the command and result

CASE STUDY: SMARTCARE CLINIC INFORMATION SYSTEM [20 MARKS]

SmartCare Clinic is a private healthcare provider that uses MongoDB to manage clinical operations and system security. The clinic stores patient appointment records and staff login activity to support reporting, monitoring, and system optimisation.

You are appointed as a junior database analyst to assist the clinic in querying data, generating reports, optimising performance, and performing data maintenance tasks.

Collections Used

appointments

Stores patient appointment and billing information.

Fields include:

- appointmentId
- patientName
- doctor
- department
- status (COMPLETED, PENDING, CANCELLED)
- fee
- appointmentDate

loginSessions

Stores staff login activity.

Fields include:

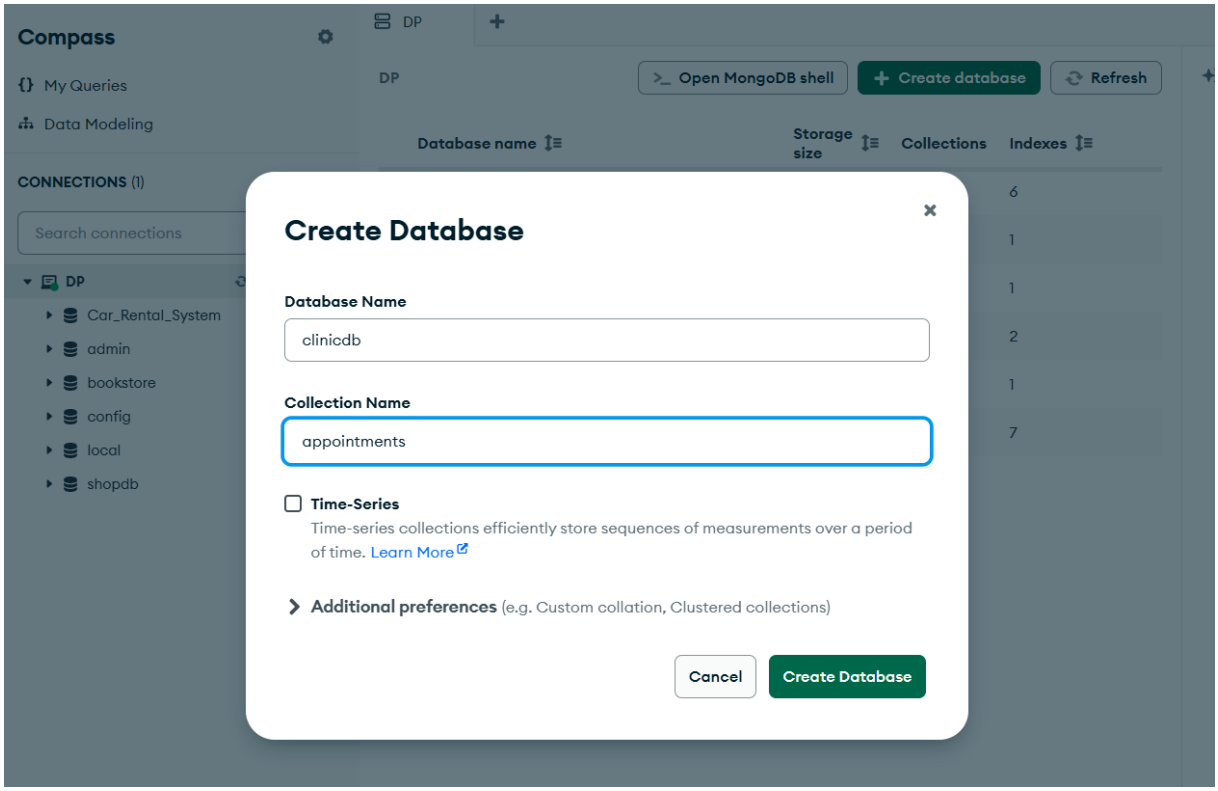
- staffId
- staffName
- role
- status (ACTIVE, INACTIVE)
- ipAddress
- loginTime

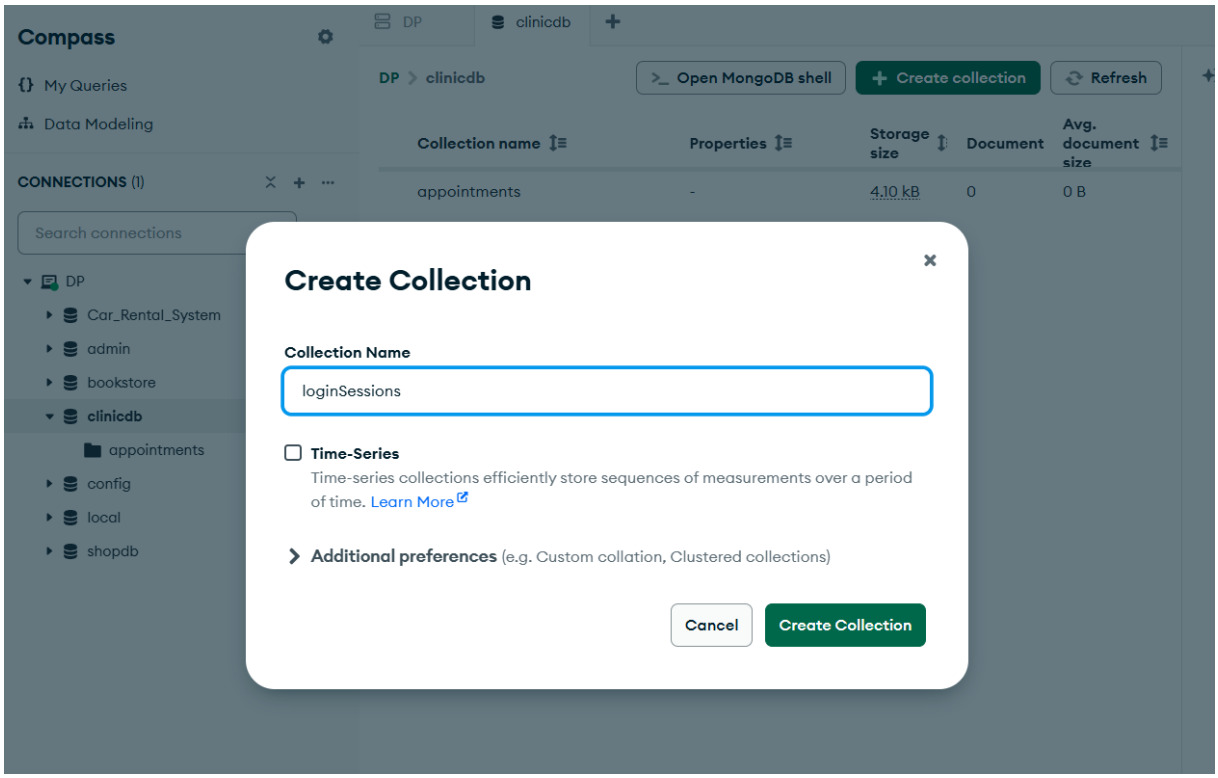
PART A - DATA PREPARATION (3 MARKS)

Question A1 (1 mark)

Create a MongoDB database named clinicdb and create the following collections:

- appointments
- loginSessions





Question A2 (2 marks)

Insert at least six (6) documents into each collection with varied and realistic data, including different statuses, departments, roles, and fees.

```
> db.appointments.insertMany([
    {appointmentID:"A001", patientName:"Jackson Wang", doctor:"Dr. Goh", department:"Cardiology", status:"COMPLETED", fee:300, appointmentDate: new D
    {appointmentID:"A002", patientName:"Lily Ong", doctor:"Dr. Ahmad", department:"Dermatology", status:"PENDING", fee:150, appointment: new Date("20
    {appointmentID:"A003", patientName:"Nur Aisyah", doctor:"Dr. Haziq", department:"Orthopedics", status:"COMPLETED", fee:320, appointment: new Date
    {appointmentID:"A004", patientName:"Siti Hamzah", doctor:"Dr. Lee", department:"Cardiology", status:"CANCELLED", fee:180, appointment: new Date("
    {appointmentID:"A005", patientName:"Daniel Lee", doctor:"Dr. Lim", department:"Dermatology", status:"PENDING", fee:150, appointment: new Date("20
    {appointmentID:"A006", patientName:"Farah Nabila", doctor:"Dr. Kumar", department:"Orthopedics", status:"CANCELLED", fee:95, appointment: new Dat
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('696d17d46f1921fe08c35542'),
      '1': ObjectId('696d17d46f1921fe08c35543'),
      '2': ObjectId('696d17d46f1921fe08c35544'),
      '3': ObjectId('696d17d46f1921fe08c35545'),
      '4': ObjectId('696d17d46f1921fe08c35546'),
      '5': ObjectId('696d17d46f1921fe08c35547')
    }
  }
```

```
> db.loginSessions.insertMany([
    { staffID:"S001", staffName:"Kai", role:"Doctor", status:"ACTIVE", ipAddress:"192.168.1.10", loginTime: new Date("2025-01-19T08:30:00") },
    { staffID:"S002", staffName:"Triumph", role:"Nurse", status:"ACTIVE", ipAddress:"192.168.1.12", loginTime: new Date("2025-01-19T12:30:00") },
    { staffID:"S003", staffName:"Izzah", role:"Nurse", status:"INACTIVE", ipAddress:"192.168.1.16", loginTime: new Date("2025-01-20T08:45:00") },
    { staffID:"S004", staffName:"Kaison", role:"Doctor", status:"INACTIVE", ipAddress:"192.168.1.25", loginTime: new Date("2025-01-20T10:30:00") },
    { staffID:"S005", staffName:"Jayne", role:"Nurse", status:"ACTIVE", ipAddress:"192.168.1.30", loginTime: new Date("2025-01-21T08:23:00") },
    { staffID:"S006", staffName:"Shawn", role:"Doctor", status:"ACTIVE", ipAddress:"192.168.1.32", loginTime: new Date("2025-01-21T09:30:00") },
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('696d1a436f1921fe08c35548'),
      '1': ObjectId('696d1a436f1921fe08c35549'),
      '2': ObjectId('696d1a436f1921fe08c3554a'),
      '3': ObjectId('696d1a436f1921fe08c3554b'),
      '4': ObjectId('696d1a436f1921fe08c3554c'),
      '5': ObjectId('696d1a436f1921fe08c3554d')
    }
  }
```

PART B - APPOINTMENT REPORTING (7 MARKS)

Question B1 - Advanced Query s Sorting (3 marks)

Retrieve appointment records that meet all of the following conditions:

- appointment status is COMPLETED
- fee is between 100 and 300 (inclusive)

Display only the following fields:

- appointmentId
- patientName
- department
- fee

Sort the results by:

- department in ascending order (A–Z)
- fee in descending order (highest first)

```
> db.appointments.find({ status:"COMPLETED", fee: {$gte: 100, $lte: 300}},
                       { _id:0, appointmentID:1, patientName:1, department:1, fee:1}).
    sort({department:1, fee: -1})
< {
    appointmentID: 'A001',
    patientName: 'Jackson Wang',
    department: 'Cardiology',
    fee: 300
  }
```

Question B2 - Aggregation Summary (4 marks)

Using MongoDB aggregation, generate a report that shows for each department:

- total number of appointments
- total fees collected

Sort the output by total fees in descending order.

```
> db.appointments.aggregate([
    {$group: { _id:"$department", totalAppointments: {$sum: 1}, totalFees: {$sum: "$fee"}}},
    {$sort: {totalFees:-1}}
  ])
< {
    _id: 'Cardiology',
    totalAppointments: 2,
    totalFees: 480
  }
  {
    _id: 'Orthopedics',
    totalAppointments: 2,
    totalFees: 415
  }
  {
    _id: 'Dermatology',
    totalAppointments: 2,
    totalFees: 300
  }
```

PART C - CLINIC SECURITY MONITORING (6 MARKS)

Question C1 - Active Staff Sessions (3 marks)

Retrieve all ACTIVE staff login sessions and display only:

- staffName
- role
- ipAddress
- loginTime

Sort the results by most recent login time first.

```
> db.loginSessions.find({status: "ACTIVE"}, {_id:0, staffName:1, role:1, ipAddress:1, loginTime:1}).sort({loginTime:-1})
< {
    staffName: 'Shawn',
    role: 'Doctor',
    ipAddress: '192.168.1.32',
    loginTime: 2025-01-21T01:30:00.000Z
  }
  {
    staffName: 'Jayne',
    role: 'Nurse',
    ipAddress: '192.168.1.30',
    loginTime: 2025-01-21T00:23:00.000Z
  }
  {
    staffName: 'Triumph',
    role: 'Nurse',
    ipAddress: '192.168.1.12',
    loginTime: 2025-01-19T04:30:00.000Z
  }
  {
    staffName: 'Kai',
    role: 'Doctor',
    ipAddress: '192.168.1.10',
    loginTime: 2025-01-19T00:30:00.000Z
  }
```

Question C2 - Distinct s Count (3 marks)

Using MongoDB operations:

a) List all distinct IP addresses used by staff with ACTIVE sessions. (1.5 marks)

```
> db.loginSessions.distinct("ipAddress", {status:"ACTIVE"})
< [ '192.168.1.10', '192.168.1.12', '192.168.1.30', '192.168.1.32' ]
```

b) Calculate the number of unique staff members who currently have ACTIVE sessions. (1.5 marks)

```
> db.loginSessions.distinct("ipAddress", {status:"ACTIVE"}).length
< 4
```

PART D - INDEXING s DATA MAINTENANCE (4 MARKS)

Question D1 - Compound Index (2 marks)

Create a compound index on the appointments collection to optimise queries that filter data by:

- department
- status

```
> db.appointments.createIndex({department:1, status:1})
< department_1_status_1
> db.appointments.getIndexes()
< [
    { v: 2, key: { _id: 1 }, name: '_id_' },
    {
      v: 2,
      key: { department: 1, status: 1 },
      name: 'department_1_status_1'
    }
  ]
```

Question D2 - Update s Delete Operations (2 marks)

a) Update all appointments with status CANCELLED to ARCHIVED. (1 mark)

```
> db.appointments.updateMany({status:"CANCELLED"}, {$set: {status:"ARCHIVED"}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 2,
    modifiedCount: 2,
    upsertedCount: 0
  }
```

b) Delete all login session records with status INACTIVE. (1 mark)

```
> db.loginSessions.deleteMany({status:"INACTIVE"})
< {
    acknowledged: true,
    deletedCount: 2
  }
```

MARKING RUBRICS

Part	Question	Assessment Criteria	Marks	
A	A1	Database clinicdb created correctly	0.5	
		Both collections (appointments, loginSessions) created	0.5	
	A2	Minimum 6 valid documents inserted into appointments	1.0	
		Minimum 6 valid documents inserted into loginSessions	1.0	
		Part A Subtotal	3.0	
B	B1	Correct filtering conditions (status COMPLETED, fee range 100–300)	1.5	
		Correct projection of required fields	0.5	
		Correct sorting (department ASC, fee DESC)	1.0	
	B2	Correct aggregation grouping by department	1.5	
		Correct aggregation calculations (count C total fees)	1.5	
		Correct sorting by total fees (DESC)	1.0	
		Part B Subtotal	7.0	
C	C1	Correct filtering of ACTIVE sessions	1.0	
		Correct projection of required fields	1.0	
		Correct sorting by latest login time	1.0	
	C2(a)	Correct retrieval of distinct ACTIVE IP addresses	1.5	
	C2(b)	Correct count of unique ACTIVE staff	1.5	
		Part C Subtotal	6.0	
D	D1	Correct compound index fields (department, status)	1.5	
		Index created successfully and shown	0.5	
	D2(a)	Correct update operation (CANCELLED → ARCHIVED)	1.0	
	D2(b)	Correct delete operation (INACTIVE sessions)	1.0	
		Part D Subtotal	4.0	
		TOTAL	20 MARKS	